FYP Plan: Two People, One Year, One Strong Product

Oct 14, 2026 · @Abdullah

No plan can guarantee success, but you can remove most of the risk. Projects usually fail for four reasons: the scope is too big, the data isn't available, there's no baseline to compare against, or nothing works end to end by the deadline. This plan is built around avoiding those.

Core principles

1. **Narrow beats broad.** "AI for healthcare" is not a project. "Classifying chest X-rays for TB screening with a lightweight model and a web interface for clinic use" is.
2. **Get a working end-to-end version by about month 5**, even if it's ugly. After that you improve it, not build it.
3. **Technical depth plus a demo.** Examiners are impressed by (a) a measurable result against a baseline and (b) something they can see working.
4. **Data decides feasibility.** Confirm the dataset exists and is accessible before you commit to any idea.
5. **Novelty can be small.** Applying a known method to a local or under-studied context, with proper evaluation, counts as a strong FYP.

Choosing your domain

Your three domains overlap, since healthcare and fintech both use ML. Judge each idea on the same five criteria, scoring 1 to 5.

Criterion	Question
Data	Is a public dataset available, or can we collect one ourselves?
Feasibility	Can two beginners build this in about 8 months?
Impact	Does a real person or institution have this problem?

Criterion	Question
Technical depth	Is there something beyond calling an API?
Demoability	Can we show it working in 5 minutes?

Cautions by domain

- **PSX stock price prediction with LSTM** is the most common FYP idea, and examiners know it. Models that "predict" prices usually just lag the previous day's price, and the results rarely hold up. If you go into finance, choose a less cliché angle.
- **Healthcare** gives great impact, but avoid anything needing restricted data (MIMIC requires credentialed access, so check first). Public sets on PhysioNet, Kaggle, and TCIA are safer. Always frame the product as decision support, not diagnosis.
- **General AI/ML** gives you the most freedom, especially in low-resource language work like Urdu and Roman Urdu, where there is real research room.

Sample ideas at the right scope

These are starting points, not final picks.

- **Healthcare:** diabetic retinopathy or skin lesion screening with explainability (Grad-CAM) and a mobile-friendly web app; an Urdu/English medical report or prescription OCR plus summarizer; ECG arrhythmia detection from PhysioNet data.
- **Finance:** sentiment analysis of PSX company announcements and financial news, linked to price movement; a fraud or anomaly detection system on transaction data with an explainable dashboard; a RAG assistant over SECP/SBP/PSX regulatory documents.
- **AI/ML:** Roman Urdu or Urdu NLP tools (such as sentiment, toxicity, or question answering); a document or invoice extraction pipeline for local small businesses; crop disease detection using local images.

Pick one idea per domain, score each, and talk to your potential supervisor before deciding.

Timeline: Phases 0 to 2

You are finishing semester 5 now. This assumes FYP-I starts in semester 7 and runs about a year. Adjust the dates to your university's calendar.

Phase 0: Now through the end of semester 5 (about 2 months) — foundations and idea shortlist

Skills (about 5 to 7 hours per week):

- Python, NumPy, pandas, matplotlib (if not already solid).
- Andrew Ng's Machine Learning Specialization or fast.ai's *Practical Deep Learning*. Pick one and finish it.
- Learn Git and GitHub. Use it from day one.
- Build one small end-to-end toy project (for example, a classifier with a Streamlit or Gradio UI). This teaches you more than any course.

Research habit:

- Start a shared Zotero library.
- Read 10 to 15 papers across your three domains, mostly abstracts and conclusions, and write 3-line notes on each.

Output: a shortlist of 3 to 5 ideas, each scored with the table above.

Phase 1: Month 1 (semester break or early semester 6) — literature review and decision

How to use Google Scholar and arXiv effectively:

1. **Find survey papers first.** Search "[topic] survey" or "[topic] review". A good survey gives you the landscape, key datasets, and open problems in one read.
2. **Chase citations.** Open a strong paper, then click "Cited by" on Scholar to see newer work building on it, and read its reference list to find the foundations.
3. **Use Papers with Code** (paperswithcode.com) for benchmarks, datasets, and implementations. It tells you what state of the art is and whether code exists.
4. **Use citation-mapping tools** like Connected Papers or Semantic Scholar to map related work.
5. **Filter for recency.** In ML, mostly read the last 3 years, except for foundational papers.
6. **Read in three passes:** abstract and figures, then intro and conclusion, then methods in detail only for the 5 to 8 papers that matter.

Read about 25 to 30 papers for your top 2 ideas, then fill out a comparison table (paper, dataset, method, metric, result, limitation).

Decision gate, by the end of the month:

- [ ] You have the dataset downloaded.
- [ ] You have found a baseline paper whose result you could reproduce.
- [ ] You can state a gap in one sentence, for example: "Existing work does X on English data, but not on Roman Urdu / not with explainability / not on a lightweight model."
- [ ] Your supervisor has approved it.

If an idea fails this gate, switch to your next one. Don't push on.

Phase 2: Month 2 — proposal and design

- Write a **1-page problem statement:** problem, who has it, existing solutions, gap, your approach, success metrics.
- Define **measurable objectives**, such as "achieve ≥ X F1 on dataset Y, outperforming baseline Z".
- Define the **scope in tiers:**
  - **Must have:** core model plus working demo.
  - **Should have:** explainability, a better UI, a comparison of 2 or 3 models.
  - **Nice to have:** deployment, mobile app, extra datasets. Drop these first if time runs short.
- Draw the **system architecture:** data → preprocessing → model → API → UI.
- Choose the **stack.** A safe default is Python, PyTorch or scikit-learn, FastAPI, and Streamlit or React for the front end, with GitHub for code and Google Colab or Kaggle for GPU.
- Split roles (see the team section) and write a **milestone calendar.**

Timeline: Phases 3 to 6

Phase 3: Months 3 to 5 — baseline and end-to-end pipeline

- **Data work (this takes longer than expected):** clean, explore, and split the data properly (train/validation/test, with no leakage), and document your EDA.
- **Reproduce the baseline** from the paper or a simple standard model. This proves your pipeline works and gives you something to beat.
- **Build a bare-bones end-to-end system:** input in, prediction out, through a basic UI. It can be ugly.

- **Milestone, end of month 5:** a working demo with a baseline model. If you have this, your project is safe.

Phase 4: Months 6 to 8 — improve, experiment, and add the impressive layer

- Run **controlled experiments:** different architectures, hyperparameters, augmentation, and an ablation study (remove one component at a time to show what helps).
- Add **one differentiating feature**, such as explainability (SHAP or Grad-CAM), a lightweight or efficient version of the model, a local-language component, or a RAG layer.
- Evaluate with the **right metrics**, not just accuracy: precision/recall/F1/AUC for medical and fraud problems; proper backtesting with no look-ahead bias for finance.
- Do **error analysis:** look at where the model fails and why. This section impresses examiners.
- Polish the UI and handle edge cases.
- If possible, **test with a real user** (a doctor, a trader, a small business owner) and record their feedback.

Phase 5: Months 9 to 10 — report, paper, and polish

- Write the report **while building, not after.** By now only the final results sections remain.
- Consider a **short paper** (a workshop, a local conference, or arXiv). Even a submitted paper strengthens your profile greatly.
- Prepare a clean GitHub repo with a README, requirements, and reproducible scripts.
- Record a **2 to 3 minute demo video** as a backup in case the live demo fails.
- Do 2 or 3 **mock vivas** with seniors or friends.

Phase 6: Months 11 to 12 — buffer

Keep this for delays, data problems, a failed experiment, report revisions, or final demo rehearsals. Projects always slip, so protect this time rather than spending it on new features.

Working as a team of two

Don't split by "one does the model, one does the rest" permanently, because both of you must be able to defend everything in the viva. A better split:

- **Person A:** data pipeline and modeling lead.
- **Person B:** backend, UI, and deployment lead.
- **Both:** literature review, experiments, and report, each reviewing the other's work.
- **Rotate** so each of you explains the other's part in a weekly 20-minute walkthrough.

Weekly rhythm:

- A 30-minute sync meeting (what was done, what's blocked, goals for next week).
- A shared task board (Notion, Trello, or GitHub Projects).
- A shared **research log**: what you tried, what happened, what you learned. This becomes the backbone of your report.
- Meet your supervisor every 2 weeks with something concrete to show.

Risk management

Risk	Prevention
Data unavailable or poor	Confirm and download it in Phase 1, before committing
Model doesn't beat the baseline	Frame the contribution as analysis or a lightweight, explainable, or localized system; honest negative results with solid analysis still pass
Scope creep	Keep the tiered scope; add Nice-to-haves only after Must-haves work
Compute limits	Use Colab/Kaggle free GPUs, smaller models, transfer learning
Team member slows down	Weekly walkthroughs and visible task boards expose problems early
Late-stage panic	The month-5 working demo and the buffer period

What examiners reward

- A clearly stated, real problem.
- A literature review that identifies a specific gap.

- A baseline comparison with numbers.
- Proper evaluation (no data leakage, the right metrics, error analysis).
- A working demo.
- You both understanding your own system deeply.

Next steps this week

- [ ] Start Zotero and a shared GitHub repo.
- [ ] Pick your ML course and set a weekly schedule.
- [ ] Search "[your topic] survey" on Scholar for each of your three domains and skim one survey each.
- [ ] Make your idea shortlist and fill out the scoring table together.
- [ ] Identify 2 or 3 potential supervisors working in these areas and look at their recent papers.